EDITH AML Intelligence Platform: Database Architecture & UML Specification (v1.0)

Project: EDITH Next-Gen AML Intelligence Platform
Target Architecture: PostgreSQL 17, .NET 10 Microservices, OpenSearch 2.x
Version: 1.0

1. Complete Database DDL (PostgreSQL 17)

```
CREATE SCHEMA IF NOT EXISTS core;
CREATE SCHEMA IF NOT EXISTS screening;
CREATE SCHEMA IF NOT EXISTS cases;
CREATE SCHEMA IF NOT EXISTS rules;
CREATE SCHEMA IF NOT EXISTS notifications;

-- ==========================================
-- SCHEMA: core
-- ==========================================
CREATE TABLE core.tenants (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE core.user_groups (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES core.tenants(id) O
```

```
    group_name VARCHAR(100) NOT NULL,
    is_active BOOLEAN NOT NULL DEFAULT true
);

CREATE TABLE core.users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES core.tenants(id) O
    group_id UUID NULL REFERENCES core.user_groups(id) ON
    username VARCHAR(100) NOT NULL UNIQUE,
    role VARCHAR(50) NOT NULL,
    is_active BOOLEAN NOT NULL DEFAULT true,
    created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE core.menus (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    parent_menu_id UUID NULL REFERENCES core.menus(id) ON
    menu_name VARCHAR(100) NOT NULL,
    path VARCHAR(255) NOT NULL,
    position INT NOT NULL
);

CREATE TABLE core.group_menu_permissions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    group_id UUID NOT NULL REFERENCES core.user_groups(id
    menu_id UUID NOT NULL REFERENCES core.menus(id) ON DE
    can_view BOOLEAN NOT NULL DEFAULT false,
    can_edit BOOLEAN NOT NULL DEFAULT false,
    can_authorize BOOLEAN NOT NULL DEFAULT false
);

CREATE TABLE core.audit_ledgers (
    id BIGSERIAL PRIMARY KEY,
    tenant_id UUID NOT NULL REFERENCES core.tenants(id) O
    actor_id UUID NULL REFERENCES core.users(id) ON DELET
    action_type VARCHAR(100) NOT NULL,
    payload JSONB NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMES
);
```

```
-- ============================================
-- SCHEMA: rules
-- ============================================
CREATE TABLE rules.country_masters (
    code VARCHAR(3) PRIMARY KEY,
    country_name VARCHAR(100) NOT NULL,
    is_high_risk BOOLEAN NOT NULL DEFAULT false,
    risk_score NUMERIC(5,2) NOT NULL DEFAULT 0.00
);

CREATE TABLE rules.nationality_masters (
    code VARCHAR(3) PRIMARY KEY,
    nationality_name VARCHAR(100) NOT NULL,
    is_high_risk BOOLEAN NOT NULL DEFAULT false,
    risk_score NUMERIC(5,2) NOT NULL DEFAULT 0.00
);

CREATE TABLE rules.aml_rules (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES core.tenants(id) O
    rule_code VARCHAR(50) NOT NULL UNIQUE,
    rule_name VARCHAR(150) NOT NULL,
    execution_mode VARCHAR(20) NOT NULL,
    rule_type VARCHAR(30) NOT NULL,
    risk_weightage NUMERIC(5,2) NOT NULL DEFAULT 0.00,
    is_active BOOLEAN NOT NULL DEFAULT true
);

CREATE TABLE rules.aml_rule_conditions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    rule_id UUID NOT NULL REFERENCES rules.aml_rules(id)
    field_name VARCHAR(100) NOT NULL,
    operator VARCHAR(20) NOT NULL,
    condition_value TEXT NOT NULL,
    logical_chain VARCHAR(5) NOT NULL DEFAULT 'AND'
);

CREATE TABLE rules.risk_rate_settings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES core.tenants(id) O
```

```
    risk_target VARCHAR(50) NOT NULL,
    benchmark_min NUMERIC(5,2) NOT NULL,
    benchmark_max NUMERIC(5,2) NOT NULL,
    assigned_risk_tier VARCHAR(20) NOT NULL
);

CREATE TABLE rules.screening_score_configs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES core.tenants(id) O
    name_weightage NUMERIC(5,2) NOT NULL,
    nationality_weightage NUMERIC(5,2) NOT NULL,
    dob_weightage NUMERIC(5,2) NOT NULL,
    id_weightage NUMERIC(5,2) NOT NULL,
    min_threshold_score NUMERIC(5,2) NOT NULL
);

CREATE TABLE rules.setup_controls (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES core.tenants(id) O
    control_code VARCHAR(50) NOT NULL UNIQUE,
    control_name VARCHAR(150) NOT NULL,
    is_enabled BOOLEAN NOT NULL DEFAULT true
);

CREATE TABLE rules.risk_assessment_masters (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES core.tenants(id) O
    factor_category VARCHAR(50) NOT NULL,
    factor_name VARCHAR(100) NOT NULL,
    weightage NUMERIC(5,2) NOT NULL
);

-- ========================================
-- SCHEMA: screening
-- ========================================
CREATE TABLE screening.customers (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES core.tenants(id) O
    customer_code VARCHAR(50) NOT NULL UNIQUE,
    customer_type VARCHAR(20) NOT NULL CHECK (customer_ty
```

```
    full_name VARCHAR(255) NOT NULL,
    dob DATE NOT NULL,
    nationality VARCHAR(3) NOT NULL REFERENCES rules.nati
    residence_country VARCHAR(3) NOT NULL REFERENCES rule
    id_type VARCHAR(50) NOT NULL,
    id_number VARCHAR(100) NOT NULL,
    risk_tier VARCHAR(20) NOT NULL DEFAULT 'LOW',
    is_blocked BOOLEAN NOT NULL DEFAULT false,
    onboarding_status VARCHAR(30) NOT NULL DEFAULT 'PENDI
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMES
    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMES
);

CREATE TABLE screening.customer_documents (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    customer_id UUID NOT NULL REFERENCES screening.custom
    document_type VARCHAR(50) NOT NULL,
    document_number VARCHAR(100) NOT NULL,
    file_path VARCHAR(255) NOT NULL,
    ocr_extracted_data JSONB NULL,
    verification_status VARCHAR(30) NOT NULL DEFAULT 'PEN
);

CREATE TABLE screening.customer_screening_history (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    customer_id UUID NOT NULL REFERENCES screening.custom
    screened_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIME
    match_status VARCHAR(30) NOT NULL,
    match_score NUMERIC(5,2) NOT NULL
);

CREATE TABLE screening.watchlist_entities (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    entity_external_id VARCHAR(100) NOT NULL,
    entity_type VARCHAR(30) NOT NULL,
    full_name VARCHAR(255) NOT NULL,
    dob DATE NULL,
    nationality VARCHAR(3) NULL REFERENCES rules.national
    source_list VARCHAR(50) NOT NULL,
    category VARCHAR(50) NOT NULL,
```

```
    is_blocked BOOLEAN NOT NULL DEFAULT true,
    is_active BOOLEAN NOT NULL DEFAULT true,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMES
);

CREATE TABLE screening.transactions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES core.tenants(id) O
    customer_id UUID NOT NULL REFERENCES screening.custom
    transaction_ref_no VARCHAR(100) NOT NULL UNIQUE,
    idempotency_key VARCHAR(100) NOT NULL UNIQUE,
    transaction_type VARCHAR(50) NOT NULL,
    amount NUMERIC(18,4) NOT NULL CHECK (amount > 0.0000)
    currency VARCHAR(3) NOT NULL,
    lcy_amount NUMERIC(18,4) NOT NULL CHECK (lcy_amount >
    sender_nationality VARCHAR(3) NOT NULL REFERENCES rul
    sender_country VARCHAR(3) NOT NULL REFERENCES rules.c
    beneficiary_name VARCHAR(255) NOT NULL,
    beneficiary_country VARCHAR(3) NOT NULL REFERENCES ru
    status VARCHAR(30) NOT NULL DEFAULT 'PENDING',
    transaction_date TIMESTAMPTZ NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMES
);

CREATE TABLE screening.screening_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    transaction_id UUID NOT NULL REFERENCES screening.tra
    pass_number INT NOT NULL,
    score NUMERIC(5,2) NOT NULL,
    match_type VARCHAR(50) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMES
);

CREATE TABLE screening.whitelists (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES core.tenants(id) O
    customer_id UUID NULL REFERENCES screening.customers(
    entity_identifier VARCHAR(255) NOT NULL,
    match_scope VARCHAR(50) NOT NULL,
    reason TEXT NOT NULL,
```

```
    expires_at TIMESTAMPTZ NULL,
    is_active BOOLEAN NOT NULL DEFAULT true
);

CREATE TABLE screening.bulk_screening_jobs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES core.users(id) ON DE
    file_name VARCHAR(255) NOT NULL,
    total_records INT NOT NULL DEFAULT 0,
    processed_records INT NOT NULL DEFAULT 0,
    status VARCHAR(30) NOT NULL DEFAULT 'PENDING'
);

CREATE TABLE screening.bulk_screening_items (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    job_id UUID NOT NULL REFERENCES screening.bulk_screen
    matched_entity_id UUID NULL REFERENCES screening.watc
    input_name VARCHAR(255) NOT NULL,
    match_score NUMERIC(5,2) NOT NULL DEFAULT 0.00,
    match_found BOOLEAN NOT NULL DEFAULT false
);

CREATE TABLE screening.delta_sync_batches (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    batch_date TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMES
    list_source VARCHAR(100) NOT NULL,
    hits_detected INT NOT NULL DEFAULT 0,
    status VARCHAR(30) NOT NULL DEFAULT 'COMPLETED'
);

CREATE TABLE screening.delta_authorizations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    batch_id UUID NOT NULL REFERENCES screening.delta_syn
    customer_id UUID NOT NULL REFERENCES screening.custom
    watchlist_entity_id UUID NOT NULL REFERENCES screenin
    match_score NUMERIC(5,2) NOT NULL,
    status VARCHAR(30) NOT NULL DEFAULT 'PENDING',
    authorized_by UUID NULL REFERENCES core.users(id) ON
);
```

```
-- ========================================
-- SCHEMA: cases
-- ========================================
CREATE TABLE cases.aml_cases (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES core.tenants(id) O
    case_number VARCHAR(50) NOT NULL UNIQUE,
    transaction_id UUID NULL REFERENCES screening.transac
    customer_id UUID NOT NULL REFERENCES screening.custom
    alert_type VARCHAR(50) NOT NULL,
    priority VARCHAR(10) NOT NULL,
    stage VARCHAR(30) NOT NULL DEFAULT 'TRIAGE',
    status VARCHAR(30) NOT NULL DEFAULT 'OPEN',
    assigned_analyst_id UUID NULL REFERENCES core.users(i
    locked_by_user_id UUID NULL REFERENCES core.users(id)
    edd_deadline TIMESTAMPTZ NULL,
    mlro_deadline TIMESTAMPTZ NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMES
    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMES
);

CREATE TABLE cases.case_rule_matches (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    case_id UUID NOT NULL REFERENCES cases.aml_cases(id)
    rule_id UUID NOT NULL REFERENCES rules.aml_rules(id)
    rule_violation_summary TEXT NOT NULL
);

CREATE TABLE cases.case_activities (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    case_id UUID NOT NULL REFERENCES cases.aml_cases(id)
    actor_id UUID NULL REFERENCES core.users(id) ON DELET
    action_type VARCHAR(50) NOT NULL,
    comments TEXT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMES
);

CREATE TABLE cases.account_freezes (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    case_id UUID NOT NULL REFERENCES cases.aml_cases(id)
```

```
    customer_id UUID NOT NULL REFERENCES screening.custom
    transaction_id UUID NULL REFERENCES screening.transac
    freeze_scope VARCHAR(30) NOT NULL,
    requested_by UUID NOT NULL REFERENCES core.users(id)
    approved_by_mlro UUID NOT NULL REFERENCES core.users(
    is_unfrozen BOOLEAN NOT NULL DEFAULT false
);

CREATE TABLE cases.goaml_filings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    case_id UUID NOT NULL REFERENCES cases.aml_cases(id)
    report_type VARCHAR(30) NOT NULL,
    report_payload_xml TEXT NOT NULL,
    validation_status VARCHAR(30) NOT NULL,
    submission_date TIMESTAMPTZ NULL
);

-- ==========================================
-- SCHEMA: notifications
-- ==========================================
CREATE TABLE notifications.notification_rules (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES core.tenants(id) O
    alert_type VARCHAR(50) NOT NULL,
    severity_tier VARCHAR(10) NOT NULL,
    target_channels VARCHAR(255) NOT NULL
);

CREATE TABLE notifications.delivery_ledgers (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    notification_rule_id UUID NOT NULL REFERENCES notific
    trace_id VARCHAR(100) NOT NULL UNIQUE,
    delivery_status VARCHAR(30) NOT NULL DEFAULT 'PENDING
    sent_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAM
);

CREATE TABLE notifications.user_shifts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES core.users(id) ON DE
    shift_start TIMESTAMPTZ NOT NULL,
```

```
    shift_end TIMESTAMPTZ NOT NULL,
    is_on_duty BOOLEAN NOT NULL DEFAULT false
);
```

2. Comprehensive Entity-Relationship Diagram (Mermaid)

```
erDiagram
    %% Core Schemas
    TENANTS ||--o{ USERS : houses
    TENANTS ||--o{ USER_GROUPS : defines
    TENANTS ||--o{ CUSTOMERS : partitions
    TENANTS ||--o{ AML_RULES : owns
    TENANTS ||--o{ WHITELISTS : authorizes
    TENANTS ||--o{ NOTIFICATION_RULES : configures
    TENANTS ||--o{ AUDIT_LEDGERS : tracks
    TENANTS ||--o{ SETUP_CONTROLS : toggles

    USER_GROUPS ||--o{ USERS : groups
    USER_GROUPS ||--o{ GROUP_MENU_PERMISSIONS : assigns
    MENUS ||--o{ GROUP_MENU_PERMISSIONS : grants
    MENUS ||--o{ MENUS : contains_submenu

    %% Rules & Masters
    NATIONALITY_MASTERS ||--o{ CUSTOMERS : assigns_nation
    COUNTRY_MASTERS ||--o{ CUSTOMERS : assigns_residence
    NATIONALITY_MASTERS ||--o{ TRANSACTIONS : sender_nati
    COUNTRY_MASTERS ||--o{ TRANSACTIONS : sender_country
    COUNTRY_MASTERS ||--o{ TRANSACTIONS : beneficiary_cou
    AML_RULES ||--|{ AML_RULE_CONDITIONS : contains
    AML_RULES ||--o{ CASE_RULE_MATCHES : violates

    %% Screening & Ingestion
    CUSTOMERS ||--o{ TRANSACTIONS : originates
    CUSTOMERS ||--o{ CUSTOMER_DOCUMENTS : owns_vault
    CUSTOMERS ||--o{ CUSTOMER_SCREENING_HISTORY : logs_ch
```

```
    CUSTOMERS ||--o{ WHITELISTS : safe_lists
    CUSTOMERS ||--o{ DELTA_AUTHORIZATIONS : flags
    CUSTOMERS ||--o{ AML_CASES : targets
    CUSTOMERS ||--o{ ACCOUNT_FREEZES : locks_entity

    DELTA_SYNC_BATCHES ||--o{ DELTA_AUTHORIZATIONS : gene
    WATCHLIST_ENTITIES ||--o{ DELTA_AUTHORIZATIONS : matc

    BULK_SCREENING_JOBS ||--|{ BULK_SCREENING_ITEMS : pro
    WATCHLIST_ENTITIES ||--o{ BULK_SCREENING_ITEMS : matc

    TRANSACTIONS ||--o| AML_CASES : triggers
    TRANSACTIONS ||--o{ SCREENING_LOGS : produces
    TRANSACTIONS ||--o{ ACCOUNT_FREEZES : freezes_remitta

    %% Cases & Lifecycles
    AML_CASES ||--|{ CASE_ACTIVITIES : logs_timeline
    AML_CASES ||--o{ CASE_RULE_MATCHES : records_trips
    AML_CASES ||--o{ ACCOUNT_FREEZES : executes_freeze
    AML_CASES ||--o{ GOAML_FILINGS : dispatches_str

    USERS ||--o{ AML_CASES : assigned_to
    USERS ||--o{ AML_CASES : locked_by
    USERS ||--o{ CASE_ACTIVITIES : performs_action
    USERS ||--o{ USER_SHIFTS : schedules

    %% Notifications
    NOTIFICATION_RULES ||--o{ DELIVERY_LEDGERS : dispatch
```

3. Core Workflow Sequence Diagram (PlantUML)

```
@startuml
autonumber
actor "Core Banking System" as CBS
participant "API Gateway (.NET 10)" as Gateway
```

```
participant "Screening Service" as Screening
database "OpenSearch 2.x" as OS
database "PostgreSQL 17" as PG
participant "Event Bus (Kafka)" as Bus
participant "Notification Engine" as Notif

CBS -> Gateway: POST /api/v1/screen/transaction (Idempote
Gateway -> PG: Check Idempotency Key & Insert PENDING Tx
Gateway -> Screening: Execute 5-Pass Screening Pipeline
Screening -> OS: Pass 2 & 3: Broad Retrieval & Phonetic S
OS --> Screening: Return Candidate Match Scores
Screening -> PG: Evaluate Thresholds & Save Screening Log

alt Critical Match / Sanction Hit (>= 90%)
    Screening -> PG: Update Tx STATUS = HELD, Create AMLC
    PG --> Gateway: Return HELD Status Response
    Gateway --> CBS: HTTP 200 OK (Status: HELD)
    Screening -> Bus: Publish TransactionHeldEvent
    Bus -> Notif: Consume Event & Dispatch P1 Alert
else Cleared
    Screening -> PG: Update Tx STATUS = CLEARED
    PG --> Gateway: Return CLEARED Status Response
    Gateway --> CBS: HTTP 200 OK (Status: CLEARED)
end
@enduml
```

4. Entity State Machine Diagram (PlantUML)

```
@startuml
[*] --> OPEN : Transaction Tripped Rule / Hit
OPEN --> EDD_ASSIGNED : Analyst Claims Case [Lock Acquire
EDD_ASSIGNED --> MLRO_REVIEW : Escalated by Analyst [SLA
EDD_ASSIGNED --> DISPOSED : False Positive Cleared
MLRO_REVIEW --> SUSPICIOUS : Confirmed STR Filing Require
MLRO_REVIEW --> DISPOSED : MLRO Dismisses Hit
SUSPICIOUS --> FROZEN : MLRO Executes Account Freeze
FROZEN --> CLOSED : goAML Regulatory XML Dispatched
```

```
DISPOSED --> [*]
CLOSED --> [*]
@enduml
```